Blok B (80 min): Docker u praksi i kontejnerizacija CLI alata

> 🎯 Ishod vježbe
>
> U osobnom repozitoriju `praktikum-<github-korisnik>` otvoren je **pull request** s grane `feature/docker` prema `main`. PR sadrži:
>
> - funkciju `read_version()` i opciju `llm --version` koja ispisuje verziju iz `version.txt`,
> - handler za SIGTERM i zadane vrijednosti konfiguracije u kodu,
> - `requirements.txt` (samo runtime) i `requirements-dev.txt` (runtime + pytest),
> - testove `tests/test_version.py` (2 nova testa uz 5 iz tjedna 2),
> - **multi-stage** `Dockerfile` (faze `base`, `test`, `runtime`) i `.dockerignore`,
> - `docker.txt` s izlazom provjera iz koraka 6.
>
> **Kriteriji prihvaćanja** (svi moraju vrijediti, nastavnik ih provjerava u PR-u):
>
> 1. `docker build --target test .` završava s `7 passed`.
> 2. `docker run --rm praktikum-llm --version` ispisuje verziju iz `version.txt`.
> 3. `docker run --rm --entrypoint id praktikum-llm` ispisuje `uid=10001(app)`, ne `root`.
> 4. `llm status` iz kontejnera javlja `Ollama is running`.
> 5. `docker stop` zaustavlja kontejner za manje od sekunde (izlazni kod `143`, ne `137`).
> 6. Poruka završnog commita i naslov PR-a slijede Conventional Commits i povećavaju **minor** verziju.
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

Sadržaj

0. Docker na računalu

1. Prvi kontejneri: ubuntu, debian-slim, alpine

2. Mreže: ime kontejnera kao DNS ime

3. Kontejnerizacija CLI alata kakav jest

4. Verzija, SIGTERM i zadane vrijednosti u kodu

5. Multi-stage Dockerfile: base, test, runtime

6. Provjere, commit i pull request

Gotovo rješenje (stanje repozitorija nakon koraka 5) nalazi se u docker-lab/blok-b/. Pretvorba alata u Python paket je zaseban blok: blok-c-vjezba.md.

0. Docker na računalu

Docker Desktop (Windows ili macOS): u *Settings* → *Resources* → *WSL integration* uključiti svoju distribuciju.

DevSecOps trenutak: članstvo u grupi `docker` ekvivalentno je root pristupu na računalu (`docker run -v /:/host ...` vidi cijeli disk). Na zajedničkom VM-u (tema 6) zato grupu `docker` ne dajemo svima.

1. Prvi kontejneri: ubuntu, debian-slim, alpine

Isti zadatak (instalirati `jq` i `git`) na tri bazne slike. Svaki `docker run --rm -it` daje nov, prazan kontejner koji se briše na izlasku.

```
docker run --rm -it ubuntu:24.04 bash
  cat /etc/os-release
  apt-get update && apt-get install -y --no-install-recommends jq git
  echo '{"course": "praktikum", "week": 3}' | jq .week
  exit

docker run --rm -it debian:trixie-slim bash
  apt-get update && apt-get install -y --no-install-recommends jq git
  exit

docker run --rm -it alpine:3.24 sh       # no bash on alpine, only sh (busybox)
  cat /etc/os-release
  apk add --no-cache jq git
  jq --version && git --version
  exit
```

Napomena: Docker Hub nema službenu sliku `ubuntu-slim`; službena `ubuntu` je već "minimalna", a *slim* varijante postoje za Debian (`debian:*-slim`) i za Python (`python:*-slim`).

Usporediti veličine:

```
docker images
```

Slika	Paketi	Okvirna veličina	Napomena
`ubuntu:24.04`	`apt`, glibc	oko 80 MB	poznato okruženje, puno paketa
`debian:trixie-slim`	`apt`, glibc	oko 75 MB	bez dokumentacije i lokalizacija
`alpine:3.24`	`apk`, musl, busybox	oko 8 MB	najmanja; `sh` umjesto `bash`, musl umjesto glibc

Prebacujemo se na **alpine**. Paketi koje smo instalirali ručno nestaju sa svakim `--rm`, pa ih zapisujemo u Dockerfile. U praznom direktoriju `~/docker-tools` napraviti `Dockerfile`:

```
FROM alpine:3.24
RUN apk add --no-cache jq git
WORKDIR /work
CMD ["sh"]
```

```
docker build -t tools:0.1 .
docker build -t tools:0.1 .              # second build: every step says
CACHED
docker history tools:0.1                 # one layer per instruction, with
its size
docker run --rm tools:0.1 jq --version   # the command after the image
replaces CMD
```

Dugoživući kontejner, `exec`, `stop` i `rm`:

```
docker run -d --name toolbox tools:0.1 sleep infinity
docker ps
docker exec -it toolbox sh               # a second process inside the same
container
  git --version; ps; exit
docker stop toolbox                      # takes about 1 s: sleep ignores
SIGTERM as PID 1 (more in step 3)
docker ps -a                             # stopped, but still exists
docker rm toolbox
```

2. Mreže: ime kontejnera kao DNS ime

```
docker network create demo
docker run -d --name web --network demo nginx:alpine
docker run --rm --network demo alpine:3.24 wget -qO- http://web:80 | head
-4   # works: Docker DNS resolves "web"
docker run --rm alpine:3.24 wget -qO- -T 3 http://web:80
# fails: default bridge has no DNS by name
docker run -d --name web-published -p 8080:80 nginx:alpine
curl -s http://localhost:8080 | head -4
# the host reaches the container via a published port
docker rm -f web web-published && docker network rm demo
```

Pravilo koje ćemo koristiti cijeli semestar: kontejneri na istoj korisničkoj mreži zovu se `http://<ime-kontejnera>:<port>`; host do kontejnera dolazi preko `-p <host-port>:<container-port>`. Isto vrijedi za Ollamu i Open WebUI u tjednu 4.

3. Kontejnerizacija CLI alata kakav jest (moduli)

Radimo u osobnom repozitoriju, na novoj grani:

```
cd ~/praktikum-$(gh api user -q .login)
git checkout main && git pull
git checkout -b feature/docker
```

U korijen repozitorija dodati prvi, jednostavni `Dockerfile` (jedna faza; u koraku 5 ga zamjenjujemo):

```
# First version: containerize llm.py + ollama_client.py as they are.
FROM python:3.13-alpine

WORKDIR /app

# Dependencies first: this layer is rebuilt only when requirements.txt
changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Source code last: it changes most often.
COPY ollama_client.py llm.py ./

ENTRYPOINT ["python", "llm.py"]
CMD ["--help"]
```

i `.dockerignore`:

```
# Keep the build context small and free of secrets.
.git
.github
.venv
__pycache__
*.pyc
.pytest_cache
.env
pytest.txt
docker.txt
```

```
docker build -t praktikum-llm .
docker run --rm praktikum-llm               # CMD --help
docker run --rm praktikum-llm status
```

Ako `ollama_client.py` zadane vrijednosti čita samo iz okoline (`DEFAULT_BASE_URL = os.getenv("OLLAMA_BASE_URL")`), `status` pada s `AttributeError: 'NoneType' object has no attribute 'rstrip'`. Razlog: `.env` je u `.dockerignore` i **nije u slici**. To je ispravno: tajne i

konfiguracija ne ulaze u sliku, nego se predaju pri pokretanju (`-e` ili `--env-file`). Popravak u kodu dolazi u koraku 4 (zadane vrijednosti u kodu), a do tada:

Docker Engine u WSL-u, Ollama u WSL-u: kontejner dijeli mrežu s hostom, pa `localhost` znači WSL:

```
docker run --rm --network host --env-file .env praktikum-llm status
docker run --rm --network host --env-file .env praktikum-llm ask "What is a
container?"
```

Docker Desktop (Windows, macOS): `localhost` u kontejneru je sam kontejner; host je `host.docker.internal`:

```
docker run --rm --env-file .env -e
OLLAMA_BASE_URL=http://host.docker.internal:11434 praktikum-llm status
```

SIGTERM i PID 1. Pokrenuti interaktivni chat u pozadini i zaustaviti ga:

```
docker run -d -it --name chat1 --network host --env-file .env praktikum-llm
chat
time docker stop chat1                    # about 1 s, then SIGKILL
docker inspect -f '{{.State.ExitCode}}' chat1   # 137 = 128 + 9 (SIGKILL)
docker rm chat1
```

`docker stop` šalje SIGTERM procesu PID 1, čeka 1 s i onda šalje SIGKILL. (Vrijeme čekanja je stop timeout kontejnera, `--stop-timeout`; na nekim instalacijama zadano je 10 s.) Python kao PID 1 nema zadani handler za SIGTERM, a kernel PID-u 1 ne primjenjuje zadanu akciju, pa signal ignorira. U produkciji to znači sporije isporuke i prekinute zahtjeve. Brzo rješenje bez promjene koda je `docker run --init` (mali init proces prosljeđuje signale); pravo rješenje je handler u kodu (korak 4).

Pogledati i veličinu: `docker images praktikum-llm`. Slika sadrži i `pytest` jer je `requirements.txt` nastao s `pip freeze` u razvojnom okruženju; to popravljamo u koraku 5.

4. Verzija, SIGTERM i zadane vrijednosti u kodu

Verzija. `version.txt` iz tjedna 1 održava release-please. Alat ga čita sam, pa se u svakom okruženju (venv, kontejner, VM) vidi koja verzija radi. U `llm.py` (gotova datoteka) dodati:

```
import signal
from pathlib import Path

VERSION_FILE = Path(__file__).with_name("version.txt")


def read_version() -> str:
    """Returns the version from version.txt (bumped by release-please), or
```

```
    0.0.0 when the file is missing."""
        try:
            return VERSION_FILE.read_text(encoding="utf-8").strip()
        except FileNotFoundError:
            return "0.0.0"
```

U `build_parser()`, odmah nakon stvaranja parsera:

```
    parser.add_argument("--version", action="version", version=f"%(prog)s
{read_version()}")
```

`Path(__file__).with_name(...)` traži datoteku pokraj `llm.py`, a ne u trenutnom direktoriju; zato `llm --version` radi i kad se alat pokrene iz drugog direktorija. (U bloku C vidjet ćemo zašto ovo prestaje raditi kad alat postane instalirani paket.)

SIGTERM. Iznad `main()` dodati handler i postaviti ga na početku `main()`:

```
def _exit_on_sigterm(signum, frame) -> None:
    # `docker stop` sends SIGTERM to PID 1. Python ignores it there unless
a handler exists.
    raise SystemExit(128 + signum)


def main() -> int:
    signal.signal(signal.SIGTERM, _exit_on_sigterm)
    args = build_parser().parse_args()
    ...
```

Zadane vrijednosti. Ako `ollama_client.py` ima `DEFAULT_BASE_URL = ""http://localhost:11434""`, zamijeniti vrijednostima iz okoline (`-e`, `--env-file`, `.env`):

```
DEFAULT_BASE_URL = os.getenv("DEFAULT_BASE_URL")
DEFAULT_MODEL = os.getenv("DEFAULT_MODEL", "llama3.2:1b")
```

Testovi. Novi `tests/test_version.py`:

```
"""Tests for reading the tool version from version.txt."""
from pathlib import Path

import llm

VERSION_FILE = Path(__file__).resolve().parent.parent / "version.txt"


def test_read_version_matches_version_file():
```

```
    assert llm.read_version() == VERSION_FILE.read_text(encoding="utf-
8").strip()


def test_read_version_falls_back_when_file_is_missing(monkeypatch,
tmp_path):
    monkeypatch.setattr(llm, "VERSION_FILE", tmp_path / "missing.txt")

    assert llm.read_version() == "0.0.0"
```

```
python -m pytest -v              # 7 passed
python llm.py --version          # llm 0.0.x
cd /tmp && python ~/praktikum-$(gh api user -q .login)/llm.py --version &&
cd -   # works from any directory
```

5. Multi-stage Dockerfile: base, test, runtime

U slici koju isporučujemo ne trebaju `pytest` ni testovi. Ovisnosti dijelimo u dvije datoteke:

```
cat > requirements.txt <<'REQ'
requests==2.34.2
python-dotenv==1.2.4
REQ
cat > requirements-dev.txt <<'REQ'
-r requirements.txt
pytest==9.1.1
REQ
pip install -r requirements-dev.txt          # local venv: everything
```

(Verzije prepisati iz svog dosadašnjeg `requirements.txt`; `pip` sam dohvaća njihove ovisnosti kao što su `urllib3` i `certifi`.)

Zamijeniti `Dockerfile` (gotova datoteka):

```
# syntax=docker/dockerfile:1
# Multi-stage build of the llm CLI: one shared base, one stage for tests,
one for the final image.
#   docker build -t praktikum-llm .               -> runtime image
(default, last stage)
#   docker build --target test -t llm-test .      -> runs the tests during
the build

ARG PYTHON_IMAGE=python:3.13-alpine

# ---- base: runtime dependencies, shared by the two stages below ----
FROM ${PYTHON_IMAGE} AS base
# PYTHONDONTWRITEBYTECODE: do not write .pyc files; they only add size to
```

```
the image layer.
# PYTHONUNBUFFERED: send stdout/stderr straight to the container log
instead of buffering,
# so `docker logs` shows output immediately (and nothing is lost if the
process is killed).
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ---- test: dev dependencies and tests; never shipped ----
FROM base AS test
COPY requirements-dev.txt .
RUN pip install --no-cache-dir -r requirements-dev.txt
COPY version.txt ollama_client.py llm.py ./
COPY tests/ tests/
RUN python -m pytest -v

# ---- runtime: application code only, running as a non-root user ----
FROM base AS runtime
RUN adduser -D -u 10001 app
COPY version.txt ollama_client.py llm.py ./
USER app
ENTRYPOINT ["python", "llm.py"]
CMD ["--help"]
```

Što je ovdje "multi-stage": `test` i `runtime` kreću od iste faze `base` (ovisnosti se instaliraju jednom i dijele cache), ali u konačnu sliku ide samo `runtime`. BuildKit gradi samo faze o kojima ciljna faza ovisi, pa obični `docker build` testove preskače, a `--target test` ih pokreće.

U `.dockerignore` promijeniti `__pycache__/*.pyc` u `**/__pycache__/**/*.pyc` (sada postoje i u `tests/`).

```
docker build --target test -t llm-test .      # 7 passed; a failing test
fails the build
docker build -t praktikum-llm .
docker images praktikum-llm                   # compare the size with the
first image
docker run --rm praktikum-llm --version
docker run --rm --entrypoint id praktikum-llm
docker run --rm --entrypoint sh praktikum-llm -c 'ls /app; pip list | grep
-i pytest || echo "no pytest"'
```

Pokretanje s Ollamom (bez `--env-file`: zadane vrijednosti su sada u kodu):

```
docker run --rm --network host praktikum-llm status                    #
Docker Engine in WSL
docker run --rm -e OLLAMA_BASE_URL=http://host.docker.internal:11434
```

```
praktikum-llm status   # Docker Desktop
docker run --rm -it --network host praktikum-llm chat
```

SIGTERM ponovno:

```
docker run -d -it --name chat2 --network host praktikum-llm chat
time docker stop chat2                          # well under a second
docker inspect -f '{{.State.ExitCode}}' chat2   # 143 = 128 + 15 (SIGTERM,
handled)
docker rm chat2
```

6. Provjere, commit i pull request

Izlaz provjera ide u `docker.txt` (kao `pytest.txt` u tjednu 2):

```
{
  echo "## docker build --target test"; docker build --target test --
progress plain --no-cache -t llm-test . 2>&1 | grep -E "passed|failed"
  echo "## version"; docker run --rm praktikum-llm --version
  echo "## user";    docker run --rm --entrypoint id praktikum-llm
  echo "## status";  docker run --rm --network host praktikum-llm status
2>&1
  echo "## stop";    docker run -d -it --name chk --network host praktikum-
llm chat >/dev/null
                     sleep 2   # let Python start and install its SIGTERM
handler
                     ( TIMEFORMAT="stopped in %R s"; time docker stop chk
>/dev/null ) 2>&1
                     docker inspect -f 'exit code {{.State.ExitCode}}' chk;
docker rm chk >/dev/null
} | tee docker.txt
```

(Na Docker Desktopu zamijeniti `--network host` s `-e OLLAMA_BASE_URL=http://host.docker.internal:11434`.) **Poruka završnog commita.** Završni commit mora prema specifikaciji Conventional Commits povećati **minor** verziju (`x.Y.z`): kad se PR spoji u `main`, release-please (tjedan 1) otvara release PR s novom minor verzijom u `version.txt`. Koji tip (prefiks) poruke to postiže, pronađite sami u specifikaciji; u naredbama ispod umjesto njega piše `<type>`. Isti prefiks koristite i u naslovu PR-a.

```
git add Dockerfile .dockerignore llm.py ollama_client.py requirements.txt
requirements-dev.txt tests/ docker.txt
git status                        # .env and .venv must NOT be listed
git commit -m "<type>: containerize the CLI with a multi-stage Dockerfile
and add --version"
git push -u origin HEAD
```

```
gh pr create --base main --title "<type>: containerize the CLI" --body
"Week 3, block B. Checks: see docker.txt"
```

Predaja (praktični zadatak 3, dio B): otvoren PR iz `feature/docker` u `main` repozitorija `praktikum-<github-korisnik>`, s datotekama iz ishoda i `docker.txt` koji potvrđuje kriterije 1–5 (kriterij 6 provjerava se iz poruke commita i naslova PR-a). PR ne spajati sami.

Dodatni zadatak: objaviti sliku u GitHub Container Registry.

```
gh auth refresh -s write:packages
gh auth token | docker login ghcr.io -u "$(gh api user -q .login)" --
password-stdin
IMAGE=ghcr.io/<org>/praktikum-$(gh api user -q .login):$(cat version.txt)
docker tag praktikum-llm "$IMAGE"
docker push "$IMAGE"
docker rmi "$IMAGE" && docker pull "$IMAGE"     # the registry now holds
the image
```

Česti problemi

Simptom	Rješenje
`permission denied while trying to connect to the Docker daemon socket`	Korisnik nije u grupi `docker`: `sudo usermod -aG docker $USER`, zatim novi terminal
`Cannot connect to the Docker daemon at unix:///var/run/docker.sock`	Daemon ne radi: `sudo systemctl start docker`; u WSL-u provjeriti `systemd=true` u `/etc/wsl.conf`; kod Docker Desktopa pokrenuti aplikaciju
`AttributeError: 'NoneType' object has no attribute 'rstrip'`	`OLLAMA_BASE_URL` nije postavljen u kontejneru: `--env-file .env` ili zadane vrijednosti u `client.py` (korak 5)
`Ollama is NOT running at http://localhost:11434` iz kontejnera	`localhost` je kontejner: `--network host` (Docker Engine) ili `host.docker.internal` (Docker Desktop)
`host.docker.internal` ne radi na Linuxu	Dodati `--add-host=host.docker.internal:host-gateway`; Ollama tada mora slušati i izvan `127.0.0.1` (`OLLAMA_HOST=0.0.0.0`), pa radije `--network host`
`ModuleNotFoundError: No module named 'llm'` u testovima	Pokretati `python -m pytest` iz korijena repozitorija; u Dockerfileu `tests/` mora biti pod `WORKDIR /app`
`llm 0.0.0` u kontejneru	`version.txt` nije kopiran u sliku: dodati ga u `COPY` u fazama `test` i `runtime`
`docker stop` i dalje čeka puni timeout (izlazni kod 137)	`ENTRYPOINT` u shell obliku (`ENTRYPOINT llm`) pokreće `sh` kao PID 1; koristiti JSON oblik `["llm"]`

Simptom	Rješenje
Svaki build ponovno instalira ovisnosti	`COPY . .` prije `pip install` poništava cache; kopirati prvo samo `requirements.txt`, kod na kraju
`exec format error`	Slika je za drugu arhitekturu (npr. `arm64` s Maca na `amd64` VM-u): `docker build --platform linux/amd64 ...`